arXiv:2510.27044v2 [cs.LG] 30 Nov 2025

Limits of Generalization in RLVR: Two Case Studies in Mathematical Reasoning

Md Tanvirul Alam
Rochester Institute of Technology
Rochester, NY, USA
ma8235@rit.edu

Nidhi Rastogi
Rochester Institute of Technology
Rochester, NY, USA
nxrvse@rit.edu

Abstract

Mathematical reasoning is a central challenge for large language models (LLMs), requiring not only correct answers but also faithful reasoning processes. Reinforcement Learning with Verifiable Rewards (RLVR) has emerged as a promising approach for enhancing such capabilities; however, its ability to foster genuine reasoning remains unclear. We investigate RLVR on two combinatorial problems with fully verifiable solutions: *Activity Scheduling* and the *Longest Increasing Subsequence*, using carefully curated datasets with unique optima. Across multiple reward designs, we find that RLVR improves evaluation metrics but often by reinforcing superficial heuristics rather than acquiring new reasoning strategies. These findings highlight the limits of RLVR generalization, emphasizing the importance of benchmarks that disentangle genuine mathematical reasoning from shortcut exploitation and provide faithful measures of progress. Code available at `https://github.com/xashru/rlvr-seq-generalization`.

1 Introduction

Large language models (LLMs) have recently advanced rapidly on mathematical and programming benchmarks [4, 5, 20, 1, 21]. A key driver is *Reinforcement Learning with Verifiable Rewards* (RLVR), which fine-tunes pretrained models against automatically checkable signals such as exact answers or unit tests [4, 22]. This paradigm eliminates reliance on human annotation, enabling scalable training on large problem sets and delivering consistent gains on challenging reasoning tasks [3].

Despite strong empirical gains, the nature of RLVR's improvements remains unclear. Studies show it often boosts accuracy while reducing exploration [19, 2, 23, 13], with base model ability acting as a ceiling [23, 19]. Gains largely reflect re-weighting existing solutions, and reasoning coverage can even contract at larger sample sizes. Standard $Pass@K$ further over-credits "lucky" completions [16], whereas stricter metrics like $CoT\text{-}Pass@K$ are more reliable but harder to scale. Reward design can also yield surprising effects: a single example can rival large-scale training [15], and even spurious rewards can drive improvements in models with strong procedural biases [7]. Overall, RLVR appears to stabilize existing competencies rather than induce new reasoning strategies.

While prior studies have offered valuable insights, many rely on benchmarks where the correctness of reasoning is difficult to verify, making it unclear whether improvements reflect genuine mathematical competence or superficial pattern matching. We address this gap by focusing on two combinatorial problems with fully verifiable solutions: *Activity Scheduling*, which admits a unique greedy optimum, and the *Longest Increasing Subsequence* (LIS), which can be solved using dynamic programming. By constructing datasets where each instance has a single optimal sequence, we can precisely measure not only answer accuracy but also sequence fidelity and structural behaviors, such as correct sorting in Activity Scheduling. This verifiable setup provides a rigorous lens on RLVR, revealing when

39th Conference on Neural Information Processing Systems (NeurIPS 2025) Workshop: MATH-AI.

Activity Scheduling Example

Determine the largest subset of activities that can be scheduled without any overlaps.

ID	Start	End
1	06:09	07:24
2	07:13	08:23
3	07:29	09:28
4	08:24	10:18
5	04:48	06:14

Ground truth: `\ids{5,2,4}`, `\answer{3}`

LIS Example

Determine the longest strictly increasing subsequence (by VALUE) from the rows below (use the row IDs).

ID	Value
1	797
2	476
3	335
4	452
5	606

Ground truth: `\ids{3,4,5}`, `\answer{3}`

Figure 1: Example question and ground-truth for Activity Scheduling (left) and LIS (right).

observed gains arise from heuristic shortcuts versus genuine reasoning strategies, and highlighting broader implications for the design of mathematical reasoning benchmarks.

2 Experimental Setup

2.1 Tasks

Activity Scheduling. Each activity i has start and finish times (s_i, f_i) with $s_i < f_i$, and intervals are half-open $[s_i, f_i)$. The goal is to select a maximum subset of non-overlapping activities [17]. Instances are constructed so that the greedy earliest-finish-time algorithm with deterministic tie-breaking yields the unique optimum, reported as IDs sorted by f_i (ties by smaller i).

Longest Increasing Subsequence (LIS). Given $a_1, \ldots, a_n \in \mathbb{Z}$, find indices $1 \le i_1 < \cdots < i_k \le n$ maximizing k with $a_{i_1} < \cdots < a_{i_k}$. Uniqueness is enforced via an $O(n^2)$ dynamic-programming count, and the LIS is reconstructed with patience sorting in $O(n \log n)$ [18].

Our generator enforces uniqueness of the optimal solution for both tasks, yielding a single ground-truth ID sequence with a fixed canonical reporting order (see Appendix A).

Dataset. We generate 2000 instances per task, half with hints, with sequence lengths 5–16. To avoid leakage, train and test use disjoint length ranges, leaving 462 test cases for Activity and 428 for LIS; the remainder form the training set. Example questions and their corresponding ground truths are illustrated in Fig. 1, with detailed prompts provided in Fig. 6 in Appendix.

2.2 Reward Functions

For each instance $(x, y^\star)$ with ground-truth answer $a^\star$ and unique optimal sequence $s^\star = (i_1, \ldots, i_L)$, the output y is parsed into $\hat{a}(y)$ (from `\answer{...}`) and $\hat{s}(y)$ (from `\ids{...}`); if parsing fails, $\hat{s}(y) = \varnothing$. All rewards lie in $[0, 1]$:

(1) Answer-only. This reward evaluates only the correctness of the final numeric answer:

$$r_{\text{ans}}(y) \;=\; \mathbb{I}[\hat{a}(y) = a^\star]\,,$$

where $\mathbb{I}[\cdot]$ denotes the indicator function, equal to 1 if the condition holds and 0 otherwise.

(2) Answer + Format (LIS only). To stabilize behavior when the policy begins omitting reasoning, we introduce a small formatting bonus, applied only to the *LIS* task(see §3.1)). Define the format indicator

$$\text{fmt}(y) \;=\; \mathbb{I}\Big[y \text{ contains } \texttt{<think>...</think>} \text{ and valid } \texttt{\answer\{...\}}, \texttt{\ids\{...\}}\Big],$$

and combine it with answer correctness using a mixing weight $\lambda = 0.1$:

$$r_{\text{ans+fmt}}(y) \;=\; (1-\lambda)\, r_{\text{ans}}(y) \;+\; \lambda\, \text{fmt}(y).$$

(3) Exact-IDs. Rewards 1 if and only if the predicted sequence matches the optimum:

$$r_{\text{ids,exa}}(y) \;=\; \mathbb{I}\big[\hat{s}(y) = s^\star\big].$$

(4) Prefix-IDs. This reward grants partial credit proportional to the length of the longest common prefix with the ground-truth sequence, while applying a small penalty if the predicted length is incorrect (clipped at 0). Define

$$m(y) \;=\; \max\{\, m \in \{0,\dots,L\} : (\hat{s}_1,\dots,\hat{s}_m) = (i_1,\dots,i_m) \,\},$$

and fix a length-penalty $\gamma = 0.1$. The reward is then

$$r_{\text{ids,pre}}(y) \;=\; \max\Big\{\, 0,\; \frac{m(y)}{L} \;-\; \gamma\, \mathbb{I}\big[\, \hat{s}(y) = \varnothing \;\vee\; |\hat{s}(y)| \neq L \,\big] \Big\}.$$

(5) Sorting-Match (Activity only). In the *Activity Scheduling* task, sorting by finish time is the first step of the greedy algorithm. Interestingly, we observe that even without explicit instructions, model responses often begin with a sorted version of the input sequence, which can be extracted reliably (see §3.3). This motivates an auxiliary reward that checks whether the extracted sorted sequence matches the ground-truth sorted order of activities:

$$r_{\text{sort}}(y) \;=\; \mathbb{I}\big[\, \hat{s}_{\text{sort}}(y) \;=\; s^{\star}_{\text{sort}} \,\big].$$

where $\hat{s}_{\text{sort}}(y)$ is the sequence of activity IDs sorted as extracted from the model output, and $s^{\star}_{\text{sort}}$ is the canonical sorted sequence by increasing finish time (breaking ties by smaller ID).

2.3 Training & Evaluation

We fine-tune *Qwen2.5-7B-Instruct* [11] with GRPO [8, 4] using the `verl` framework [10]. Unless noted, max generation length is $T_{\max} = 2048$ (extended to 7680 for LIS with $r_{\text{ans+fmt}}$). Each PPO update uses 256 prompts with 8 rollouts from `vLLM` [6], trained for 20 epochs (120 updates) at learning rate 10^{-6} and no KL penalty ($\beta_{\text{KL}} = 0$). Training prompts match evaluation format.

Evaluation protocol. For each instance x we draw $k = 256$ samples $\{y_j\}_{j=1}^{256}$ with temperature 0.6 and top-$p = 0.95$, and parse each y_j into $(\hat{a}(y_j), \hat{s}(y_j))$ as in the reward definitions. We evaluate two complementary notions of accuracy: Acc_{ans} (correctness of the reported cardinality/length) and Acc_{ids} (exact match of the ID sequence). Both are measured under *Pass@k* and *Self-consistency (SC)*:

- **Pass@k** [23]. Success under a k-sample budget, defined by whether at least one of the k generations is correct:

$$\text{Pass@k}_{\text{ans}}(x) = \mathbb{I}\big[\exists\, j \le k :\; \hat{a}(y_j) = a^{\star}\big], \qquad \text{Pass@k}_{\text{ids}}(x) = \mathbb{I}\big[\exists\, j \le k :\; \hat{s}(y_j) = s^{\star}\big].$$

- **Self-consistency (SC)** [14]. Agreement between the majority prediction aggregated over k generations and the ground truth:

$$\tilde{a}_k(x) := \text{mode}\{\hat{a}(y_j)\}_{j=1}^{k}, \qquad \tilde{s}_k(x) := \text{mode}\{\hat{s}(y_j)\}_{j=1}^{k},$$

  with deterministic tie-breaking (numerically smallest for answers; lexicographic for sequences). We report

$$\text{SC}_{\text{ans}}(x;k) = \mathbb{I}\big[\tilde{a}_k(x) = a^{\star}\big], \qquad \text{SC}_{\text{ids}}(x;k) = \mathbb{I}\big[\tilde{s}_k(x) = s^{\star}\big].$$

All metrics are averaged over the test set. Because each instance admits a unique ground-truth answer $a^{\star}$ and sequence $s^{\star}$, these definitions are unambiguous. We plot the full curves $\{\text{Pass@k}\}_{k=1}^{256}$ and $\{\text{SC}(\cdot;k)\}_{k=1}^{256}$, and also report their values at $k = 256$.

3 Results & Analysis

3.1 Training with Exact Answer Reward

Fig. 2 compares the base model and the RLVR-trained policy on *Activity Scheduling* and LIS tasks.

Activity Scheduling. Since the target is a small integer (maximum schedule length 16), the base model quickly saturates to *Pass@k* ≈ 1.0 as k increases. While the RLVR-trained policy attains